DEVOPS & CLOUD ARCHITECTURE REPORT

Auto-Incident to Agile Ticket Pipeline (DevOps Incident Watchdog)

Continuous Incident Triaging, Cloud Forensics, and Zero-Touch Agile Backlog Synchronization

STUDENT	REGISTRATION NO.	INSTITUTION	COURSE
S Hemal	24MIS0050	VIT Vellore Campus	Agile Dev Process & DevOps

AWS REGION
ap-southeast-2 (Sydney)

Executive Summary: This project delivers an autonomous Site Reliability Engineering (SRE) and Agile feedback loop deployed on AWS. When unexpected runtime exceptions occur in production (e.g. e-commerce flash sale stock depletion triggering an HTTP 500 error), AWS CloudWatch and an AWS Lambda Watchdog automatically capture the stack trace, securely archive a tamper-proof forensic JSON dump in Amazon S3, and generate a standardized Agile Bug User Story with Acceptance Criteria on GitHub Issues. Upon committing a fix with smart syntax (`Fixes #1`), GitHub Actions runs automated unit tests, deploys the updated microservice, and automatically transitions the Agile ticket to Done.

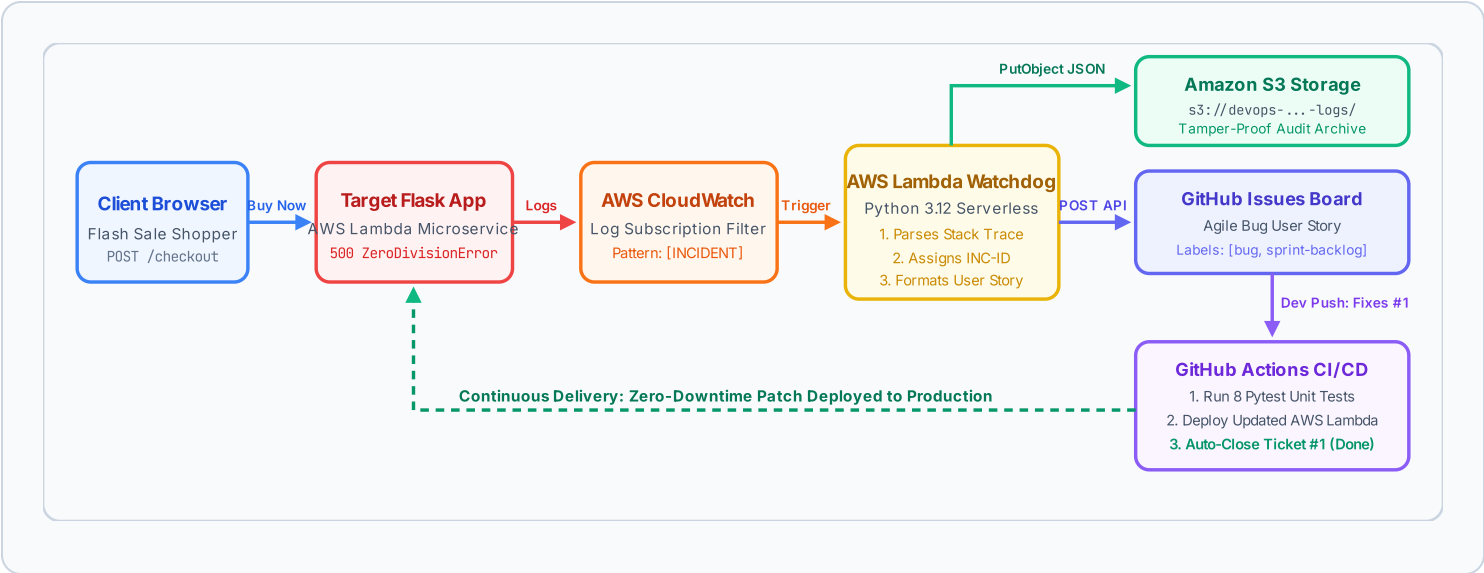
1. Target Application & Incident Failure Scenario

Target Application: A production e-commerce retail store (**Flask Microservice** with WSGI serverless adapter). The catalog features high-velocity electronics: Pro Laptop, Wireless Headphones, and a **Flagship Smartphone** set up for a promotional flash sale.

The Failure Mechanism: When flash sale traffic depletes the smartphone inventory to `0`, the legacy checkout calculation worker attempts an unhandled inventory ratio division: `discount = total / remaining_stock`. Since `remaining_stock == 0`, Python raises an unhandled `ZeroDivisionError: division by zero`, crashing the process with an **HTTP 500 Internal Server Error**.

The Patched Behavior: The corrected release performs defensive boundary checks, returning an `HTTP 409 Conflict` ("Item is Sold Out") without crashing the server.

2. Process Architecture & Component Flow


Client Browser
Flash Sale Shopper
POST /checkout
Buy Now
Target Flask App
AWS Lambda Microservice
500 ZeroDivisionError
Logs
AWS CloudWatch
Log Subscription Filter
Pattern: [INCIDENT]
Trigger
AWS Lambda Watchdog
Python 3.12 Serverless
1. Parses Stack Trace
2. Assigns INC-ID
3. Formats User Story
PutObject JSON
Amazon S3 Storage
s3://devops-...-logs/
Tamper-Proof Audit Archive
POST API
GitHub Issues Board
Agile Bug User Story
Labels: [bug, sprint-backlog]
Dev Push: Fixes #1
GitHub Actions CI/CD
1. Run 8 Pytest Unit Tests
2. Deploy Updated AWS Lambda
3. Auto-Close Ticket #1 (Done)
Continuous Delivery: Zero-Downtime Patch Deployed to Production

3. Detailed End-to-End Incident Lifecycle ("What All Happens")

Step / Phase	Component	Technical Process	Realized Output / Proof
Step 1: Checkout Crash	Target Flask App (AWS Lambda / Port 5000)	Shopper purchases the last smartphone. Stock reaches 0. Checkout calculates discount ratio with zero divisor, raising `ZeroDivisionError`.	HTTP 500 Internal Server Error & Structured JSON error output.
Step 2: Log Interception	AWS CloudWatch	CloudWatch Log Subscription filter continuously scans stdout for pattern `[INCIDENT_DETECTED] Type=ZeroDivisionError`.	Gzip-compressed base64 log event emitted to Lambda.
Step 3: Autonomous Triage	AWS Lambda Incident Watchdog	Serverless function uncompresses event, parses stack trace, assigns a unique incident ID (e.g. `INC-1791945923-07CD12B5`), and records UTC timestamp.	Structured incident dictionary with environment context.
Step 4: Forensic Archival	Amazon S3 Bucket (Private)	Watchdog uploads immutable JSON crash dump to `s3://devops-incident-watchdog-logs-.../incidents/YYYY-MM-DD/`.	Auditable 369 B JSON crash dump safely preserved.
Step 5: Agile Ticket Generation	GitHub REST API v3	Lambda executes HTTP POST to `/repos/{owner}/{repo}/issues` via Personal Access Token, injecting a User Story with Acceptance Criteria.	New issue (e.g. **#1**) created on GitHub Sprint Board tagged `bug`, `incident`.
Step 6: Automated Testing & CI/CD	GitHub Actions CI/CD	Developer fixes zero-stock check and pushes commit: `git commit -m "Fix inventory edge case. Fixes #1"`. CI/CD runs 8 `pytest` unit tests.	Green checkmark (✅ All unit tests passing in pipeline).
Step 7: Zero-Touch Close & Deployment	Git Smart Commits & AWS CLI	GitHub Actions deploys new Lambda package to AWS, and the `Fixes #1` directive closes the Agile ticket automatically without human intervention.	Issue #1 moves to **Done**; live store reports `200 OK (Healthy)`.

4. Agile Backlog Structure & Acceptance Criteria

📋 Automatically Generated Agile Bug User Story

Title: `[INCIDENT] ZeroDivisionError: division by zero in calculation worker`

Sprint Backlog Labels: bug incident sprint-backlog

User Story Description: *"As an Agile Development Team member, I need to resolve the runtime failure in `/aws/watchdog-target-app` so that high system availability is restored and customers experience zero disruption."*

Acceptance Criteria (Definition of Done):

- ☑ **Root Cause Analysis (RCA):** Crash logs inspected in Amazon S3.
- ☑ **Automated Regression Test:** Unit test in `tests/test_app.py` validating 0-inventory behavior.
- ☑ **Traceable Git Commit:** Commit message using smart syntax (`git commit -m "Fixes #1"`).
- ☑ **Continuous Integration:** GitHub Actions pipeline executes and passes all test suites.
- ☑ **Production Verification:** `/health` endpoint confirms `HTTP 200 OK, status: healthy` .

5. Faculty Viva Cheat Sheet (Quick Answers for S Hemal)

Q1: Why do we use Amazon S3 instead of just keeping logs on the server?

"Because production servers can crash, restart, or scale to zero. Amazon S3 provides an immutable, highly durable, external audit trail for compliance and post-mortem analysis."

Q2: How does GitHub get the ticket without any human clicking 'New Issue'?

"The AWS Lambda Watchdog executes upon incident detection and issues an authenticated HTTP POST request to the GitHub REST API (`/repos/.../issues`) containing the formatted User Story and Acceptance Criteria."

Q3: What makes this an Agile DevOps project rather than just ordinary monitoring?

"Standard monitoring just sends alerts to an inbox. Our pipeline bridges SRE with Agile by translating production defects directly into actionable Sprint Backlog User Stories with an explicit Definition of Done."

Q4: How does the Agile ticket close automatically?

"Through GitHub Smart Commits: our developer references `Fixes #1` in the commit message. Once GitHub Actions CI/CD passes all unit tests and deploys the fix, GitHub automatically closes the issue."